EBU6304 Software Engineering Group Project

2025-26 Final Short Report

Group number: XXX Project: TA Recruitment System

Group number	XXX	TA Recruitment System
member 1	QM no: 231222431	Name: Yuhang Fu
member 2	QM no: 231222682	Name: Jiajin He
member 3	QM no: 231222257	Name: Ruiqi Guo
member 4	QM no: 231220172	Name: Shuai Na
member 5	QM no: 231222073	Name: Leilei Tao
member 6	QM no: 231222453	Name: Yi Zheng

Design

Our project is a small recruitment system for teaching assistant posts. We built it around the three users who would normally touch this process: TA applicants, module organisers (MO), and HR/admin reviewers. Since this was a coursework prototype, we kept the technology stack simple with JSP, servlets and JSONL files, but we still tried to make the responsibility of each role clear.

Architecture.

• Presentation layer: JSP pages and shared CSS/JavaScript provide role-specific interfaces for login, job browsing, application submission, MO screening, and HR review.

• Controller layer: Jakarta Servlets expose JSON APIs including /jobs, /applications, /mo/jobs, /mo/candidates, /admin/workload, and AI endpoints.

• Service layer: JobService, ApplicationService, AttachmentService, WorkloadService, AiService, and Auth-related services implement validation, status transitions, matching, and workload logic.

• Repository/storage layer: lightweight JSONL repositories persist jobs, users, applications, and attachments, giving deterministic demo data while avoiding a database dependency.

Design concern	Decision	Reason
Role separation	TA, MO, and ADMIN sessions control page access and API permissions.	Prevents accidental cross-role actions and keeps workflows understandable.
Application review	MO review is job-first, with status filters and candidate drill-down.	MO users normally assess candidates in the context of a module vacancy.
AI support	AI outputs are advisory summaries and match insights, backed by rule-based fallback.	The system remains usable without external API configuration.
File handling	CV and transcript are uploaded, stored, and exposed as direct attachment links.	Reviewers need concise evidence access during screening.

Design principles and patterns.

• MVC-style separation is used: JSP/JS renders views, servlets coordinate requests, services own business rules, and repositories manage persistence.

• Validation is performed both client-side for usability and server-side for correctness, especially around role checks, job status, file requirements, and duplicate applications.

• Progressive enhancement is used on the frontend: static pages remain understandable, while JavaScript loads live data and updates UI state.

• Consistent visual language is applied through shared button, panel, border, status tag, and background styles so TA, MO, and HR pages feel like one product.

Data model and workflow design.

We kept the data model small so that it was easy to debug during the group project. A User record stores identity, role, display name, email, skills, workload and resume text. A JobPosting record stores the module, title, slot count, creator MO, status and skill requirements. An Application record links one TA candidate to one job and records the review status, submitted time and attachment count. Attachment records store uploaded evidence and extracted text. This was enough to follow a decision from the job, to the candidate, to the uploaded files.

Workflow	Main steps	Important states
TA application	TA signs in, browses open jobs, opens detail page, uploads CV/transcript, confirms submission.	Not started, ready to submit, submitted/disabled, visible in My Applications.
MO screening	MO creates job, opens job candidates, filters applications, reads detail, reviews evidence, updates decision.	Pending, interviewed, accepted, rejected, all.
HR monitoring	HR selects candidate/job, runs AI support, checks workload dashboard and candidate distribution.	No AI result, analysed, workload normal, workload high.

Security and permission considerations.

• Role checks are performed before returning privileged MO or HR data, rather than relying only on hidden navigation links.

• MO ownership is enforced so an MO can manage jobs created by themself, while TA users can still browse open jobs across modules.

• Submission logic includes duplicate protection because frontend disabling alone is not sufficient if requests are repeated or refreshed.

• Attachment access is exposed through direct reviewer links, but the design assumes authenticated review sessions rather than anonymous public download.

Implementation

We built the system in stages instead of trying to finish every role at once. The first stage was the servlet/JSP skeleton, login flow and JSONL repositories. After that, the TA job list and application submission were added. MO job creation and candidate screening came next, followed by the HR workspace, workload dashboard, AI support and final UI cleanup.

Build area	Implemented features
Authentication	Role selection, demo login, TA self-registration, internal MO/HR account assumption, session redirect logic.
TA workflow	Live job list, real pagination, job detail, file upload, submission confirmation, duplicate-submission prevention, application history.
MO workflow	Job creation, job-scoped application review, status filtering, approve/reject/mark interviewed actions, candidate details, attachment links, AI summary.
HR workflow	Candidate/job selection, AI assessment rendering, workload dashboard with applicant display names and overload warnings.
AI integration	OpenAI chat provider with environment configuration and deterministic rule-based fallback for stable demos.

Version control and collaboration.

• Git branches were used for feature ownership, allowing each member to work independently before merging into the shared project branch.

• Commits were kept task-oriented, for example separating login/auth changes, MO workflow improvements, UI unification, and live-data fixes.

• Merge conflicts were resolved by preserving working features while aligning shared files such as JSP pages, CSS, JavaScript, services, and JSONL seed data.

• Pull requests supported code review and helped identify integration issues such as stale demo data, frontend cache versions, and inconsistent UI behaviour.

Build plan and integration sequence.

Phase	Goal	Main output	Integration risk handled
1. Foundation	Create the Maven web project, JSP routing, static assets and shared layout.	Runnable servlet/JSP application with login entry point.	Established common file structure before feature branches diverged.
2. Domain services	Implement user, job, application and attachment services over JSONL repositories.	Reusable backend APIs and deterministic seed data.	Reduced duplicated business logic in JSP pages.
3. TA flow	Allow candidates to browse jobs, view details, upload evidence and submit once.	Live job list, application form, My Applications page.	Fixed fake pagination and duplicate submit behaviour.
4. MO flow	Allow module organisers to create jobs and review candidates by status.	MO dashboard, candidate detail, approve/reject/interview actions.	Aligned MO-owned jobs with applications submitted from TA accounts.

Phase	Goal	Main output	Integration risk handled
5. HR/AI flow	Provide cross-job insight, AI decision support and workload monitoring.	Admin workspace, AI summary cards, workload dashboard.	Mapped candidate IDs to display names for readability.
6. Polish	Unify page backgrounds, buttons, status labels, typography and disabled states.	Consistent TA/MO/HR UI and clearer acceptance behaviour.	Removed visual inconsistency that made workflows feel disconnected.

Implementation trade-offs.

• JSONL storage was selected because it is easy to inspect, version and reset for demonstrations; a production version would migrate the repositories to a relational database.

• JSP was kept as the rendering technology to match the coursework stack, while JavaScript was used where dynamic data loading made the user experience clearer.

• The AI provider is isolated behind AiService so the application can fall back to deterministic rule-based analysis when API keys or network access are unavailable.

• The UI uses shared CSS tokens rather than a component framework, which kept the build lightweight but required careful manual consistency checks.

Testing

Testing was a mixture of build checks, API checks and repeated browser testing. The browser tests mattered most because many bugs only appeared when data moved from one role to another, for example from a TA submission into the MO review page.

Test technique	Example test focus	Result or use
Unit tests	Service-level behaviour such as workload calculation, application submission, and AI scoring.	Verified isolated business rules where dependencies can be mocked or seeded.
Integration tests	TA application lifecycle, MO status update, and repository persistence.	Checked that controllers, services, and repositories cooperate correctly.
API testing	GET /jobs, POST /applications, GET /mo/candidates, PUT /mo/applications, GET /admin/workload.	Confirmed payloads, status codes, and role-sensitive data returned by endpoints.
Browser acceptance	TA submit confirmation, disabled submitted button, MO review filters, HR workload names, clickable attachments.	Caught UI issues that automated backend tests could not see.
Build checks	Maven package with tests skipped where legacy tests had outdated constructors.	Verified main source compiles and WAR packaging succeeds.

Specific examples.

• Duplicate application prevention was tested by submitting the same TA/job combination multiple times. The UI disables the button after success, and the backend returns the existing application instead of creating a duplicate.

• MO review was tested by creating a job as an MO, applying as a TA, and checking that the created job and application appear in the appropriate MO review workspace.

• HR workload display was tested by comparing applicant identifiers against user profiles and verifying that the UI shows names such as Alice Demo rather than only ta-001.

• Attachment presentation was tested by uploading CV/transcript files and confirming that the MO detail page presents concise, directly clickable file links.

Acceptance test checklist.

Area	Manual verification steps	Expected result
Login and roles	Log in as TA, MO and HR/admin demo users from index.jsp.	Each role lands on the correct workspace and sees only relevant navigation.
TA application	Open jobs.jsp, use pagination, select a job, upload both files and submit.	Confirmation appears; submit button becomes disabled; the application appears in My Applications.
MO review	Log in as the MO that created a job, open mo.jsp and filter by each status.	Owned jobs load immediately; filters show Pending, Interviewed, Accepted, Rejected and All correctly.
MO detail	Open a candidate detail page and inspect attachments and AI summary.	Files are presented as direct readable links; review actions require confirmation.
HR dashboard	Open admin.jsp, run AI analysis and scroll to Workload Dashboard.	AI text is readable; workload table shows applicant names rather than only IDs.

Area	Manual verification steps	Expected result
Visual consistency	Compare TA, MO and HR pages after cache refresh.	Background, buttons, borders, status tags, text sizes and disabled states follow the same style system.

Defect-driven testing.

Several checks were added only after we saw real integration bugs. For example, one JSP page failed because JavaScript template literal syntax was accidentally parsed as server-side EL. After fixing it, we reloaded the TA job list and checked the pagination again. Another bug was that MO data appeared only after clicking a filter, so we started testing the initial page-load state separately from filter changes. This made the final acceptance testing more practical.

Known testing limitation: some existing test sources lag behind newer JobPosting and JobService constructor signatures. The main application build was therefore verified with **mvn -Dmaven.test.skip=true package**, while the outdated tests should be updated before final continuous-integration use.

The use of Generative AI (GenAI)

We used GenAI in two places. First, it is part of the product: the HR/MO pages can produce a candidate-job summary, missing-skill notes and a match explanation. Second, we used GenAI during development for drafting code changes, checking error messages, preparing test checklists and improving this report. We treated those outputs as drafts, not as final answers.

Stage	How GenAI supported the work	Critical evaluation
Requirements/design	Suggested possible TA, MO and HR workflow states.	Useful for brainstorming, but we still had to decide what matched our actual coursework scope.
Implementation	Helped draft JSP/JS/Servlet changes and repeated UI patterns.	Good for quick edits, but it sometimes missed local context, such as JSP parsing JavaScript syntax.
Testing/debugging	Helped read stack traces and turn bugs into browser checks.	Helpful for diagnosis, but the result still had to be tested in the running web app.
Documentation	Helped turn notes and commits into report sections.	Useful for structure, but the group needed to check that the wording matched what we actually built.

Limitations and challenges.

• GenAI can produce plausible but wrong assumptions about the current codebase; every code change therefore had to be checked against actual files and browser behaviour.

• Generated frontend code may conflict with JSP syntax or server-side expression parsing, so template-language awareness is essential.

• AI-assisted recommendations in the product should remain decision support rather than automatic hiring decisions, because fairness, explainability, and data quality require human judgement.

• The rule-based fallback is important because relying entirely on external APIs would make demonstrations unstable when API keys or network access are unavailable.

Responsible AI design.

The AI result is only a support tool. MO and HR users still need to read the files, check the candidate profile, consider workload and make the final decision themselves. This matters because a score based on keywords can miss teaching ability, availability, communication skills or module-specific context. For that reason, the page keeps the original evidence visible next to the AI summary.

Project risks and future improvements.

• Persistence risk: JSONL files are easy for coursework but not safe for concurrent production writes; future work should use a database with transactions.

• Testing risk: legacy tests should be updated to match the current constructors and added to continuous integration before deployment.

• Security risk: attachment access and role permissions should be hardened with stronger server-side checks and audit logs in a production version.

• AI risk: generated analysis should include confidence, evidence links and bias monitoring before being used in real recruitment decisions.

• Usability improvement: future versions could add notification emails, richer application timelines and bulk MO review actions.

Requirements traceability.

The final system was checked against the main stakeholder goals rather than only against individual pages. TA users need to discover relevant vacancies, submit evidence once, and understand the state of each application. MO users need to create jobs,

identify candidates for their own modules, and make clear review decisions. HR users need a higher-level overview of candidate suitability and workload distribution. The implementation maps each of these needs to a visible screen, a backend endpoint and a verification action.

Stakeholder need	Implemented support	Verification evidence
TA can find jobs	jobs.jsp loads live open jobs from the backend and supports real pagination.	Create a new MO job, log in as TA and confirm it appears in the job list.
TA can submit once	apply.jsp requires files, asks for confirmation, then disables the submit action after success.	Submit an application twice and confirm no duplicate record is created.
MO can review own jobs	mo.jsp loads MO-owned jobs immediately and exposes status filters and candidate cards.	Log in as an MO and verify owned jobs appear without needing to click All.
MO can inspect evidence	candidate detail pages show skills, resume text, attachments and AI summary controls.	Open the candidate detail page and click attachment links directly.
HR can monitor workload	admin.jsp shows AI decision support and workload rows with display names.	Run the HR dashboard and confirm names replace raw user IDs.

API and data-flow summary.

The frontend is intentionally thin: JSP supplies page structure and JavaScript fetches JSON from servlet endpoints. This reduces server-rendered duplication and makes the UI easier to refresh after actions. For example, the TA job list calls the jobs endpoint, the apply page posts a multipart application request, and the MO dashboard refreshes candidate cards after every review action. The same principle is used in HR where the workload table and AI analysis panels are loaded dynamically.

Endpoint area	Purpose	Frontend consumer
Authentication/session	Checks current user role and redirects to the correct workspace.	index.jsp and role-specific navigation guards.
Jobs	Returns open jobs for TA users and MO-owned jobs for organisers.	jobs.jsp, job-detail.jsp and mo.jsp.
Applications	Creates TA applications, prevents duplicates and returns application history.	apply.jsp and applications.jsp.
MO review	Returns job candidates and updates application status.	mo.jsp and mo-candidate-detail.jsp.
HR/admin	Returns candidates, jobs, workload rows and AI decision-support payloads.	admin.jsp.

User interface strategy.

A major implementation challenge was that different members initially built different pages with slightly different visual rules. The final UI pass therefore treated the interface as one design system. The HR background was reused across role pages except login, the TA Home navigation item was removed where it duplicated the landing flow, and TA/MO/HR exit links were aligned to a small arrow plus Exit label. Buttons share the same gradient primary style, outline secondary style and grey disabled state. Status tags use consistent semantic colours: pending is warm yellow, interviewed is blue, accepted is green and rejected is red.

Deployment and running assumptions.

• The application is packaged as a Maven WAR and run locally on a servlet container such as Tomcat/Jetty for demonstration.

• The data directory stores JSONL files for users, jobs, applications and attachments; these files should be reset or backed up before formal demos.

• The frontend may cache JavaScript and CSS, so cache-version query strings were used during UI fixes to make browser verification reliable.

• External AI configuration is optional: if the OpenAI provider is not configured, the rule-based fallback still produces deterministic review support.

Additional quality evidence.

Many of our bugs were not pure Java errors; they were interaction problems. Examples included a file-upload button whose selected state was unclear, a TA submit button that could be clicked more than once, fake pagination controls, MO data that appeared only after a filter click, and HR workload rows showing IDs instead of names. We checked these fixes in the browser after refreshing or rebuilding the local server.

Validation rules and edge cases.

Rule or edge case	Handling in the system	Reason
Missing CV or transcript	The TA application form blocks submission and shows a clear file-specific error.	MO reviewers need evidence before making a fair decision.

Rule or edge case	Handling in the system	Reason
Repeated submit click	The frontend disables the submit button and the backend checks for an existing TA/job application.	Prevents duplicate applications even if a browser sends repeated requests.
MO without owned jobs	The MO dashboard explains that no owned jobs are available instead of showing unrelated jobs.	Prevents role confusion and protects job ownership boundaries.
New MO-created job	Open jobs are loaded from the backend so TA users can see newly created vacancies.	Avoids hard-coded demo listings and supports real workflow integration.
AI unavailable	The rule-based provider returns deterministic summaries and missing-skill suggestions.	Keeps the HR/MO review demo reliable without external API dependency.
Long file names	Attachment rows are simplified into readable direct links instead of dense metadata strings.	Makes candidate evidence easier to open during review.

Detailed browser test scenarios.

The final acceptance pass was organised around role-switching scenarios. In scenario one, a TA logs in, opens the job list, confirms that the list is populated from live backend data, moves between pages, selects a job, uploads a CV and transcript, confirms submission, and verifies that the submit button changes to a disabled completed state. In scenario two, the MO who owns the job logs in and confirms that the new application is visible without pressing an extra filter button. The MO opens candidate detail, checks the applicant profile, opens the attached files, then approves, rejects or marks the candidate interviewed after a confirmation prompt. In scenario three, HR opens the admin workspace, runs AI analysis, checks that the result is displayed as readable cards rather than raw JSON, and verifies that the workload table uses candidate names. These scenarios were more valuable than isolated page checks because they follow the same data as it moves across roles.

Maintainability considerations.

• Service classes centralise workflow rules, which makes it easier to change review logic without editing every JSP page.

• Shared CSS variables and page classes reduce the cost of later visual changes across TA, MO and HR pages.

• Cache-versioned JavaScript references helped the team verify the newest frontend code during rapid local testing.

• Separating AI provider logic from the UI means future providers or scoring algorithms can be added without rewriting the HR page.

• Using display names alongside IDs improves readability while preserving stable identifiers for internal records.

Evaluation of final outcome.

By the end, the prototype covered the main recruitment path: an MO can create a job, a TA can apply, the MO can screen the application, and HR can review workload and AI support information. The biggest improvement compared with the early version was that data moved between roles instead of sitting in separate demo pages. The weakest part is still the prototype infrastructure: JSONL files, limited automated tests and simple file security. For the coursework brief, however, it demonstrates the design, build plan, testing approach and GenAI use clearly.

Team reflection.

The project reminded us that integration is harder than building a page in isolation. Early versions looked complete because TA, MO and HR all had screens, but testing showed that the data did not always travel between them. We then changed our checking style: instead of asking whether a page existed, we asked whether a full user journey worked. That shift made the final system much stronger.

Lessons for future work.

• Define shared UI tokens earlier so feature pages do not drift visually during parallel development.

• Create end-to-end test data for each role at the beginning instead of relying on hard-coded demo records.

• Update automated tests whenever constructor signatures or service responsibilities change.

• Document API contracts while implementing endpoints so frontend and backend assumptions stay aligned.

• Keep AI outputs explainable and reversible because recruitment decisions require human accountability.

Project management and coordination.

The team divided work by role area, but the final product required frequent coordination across those areas. The TA workflow depended on job data created by MO users. The MO workflow depended on applications and attachments submitted by TA users. The HR workflow depended on both candidate profile data and review/workload data. Because of these dependencies, the team used integration checkpoints rather than waiting until the end to combine all work. When a member changed a shared model, endpoint or CSS rule, the affected pages were rechecked so that the change did not silently break another role.

Coordination issue	Impact if unmanaged	Mitigation used
Shared data model changes	Old pages or tests may call constructors or fields that no longer exist.	Review model changes together and rebuild the project after integration.

Coordination issue	Impact if unmanaged	Mitigation used
Frontend cache	Browser may show old JavaScript or CSS after a fix.	Use cache-version query strings and hard refresh during verification.
Demo seed data	Pages may appear to work only because hard-coded records match one account.	Test with newly created jobs and newly submitted applications.
Role permissions	A fix for TA visibility may accidentally expose MO-only data.	Verify each role separately after changing endpoints.
UI consistency	Different pages may look like separate systems.	Centralise colours, button states, borders, backgrounds and status tags.

This coordination approach improved both the software and the team process. It made hidden assumptions visible, especially assumptions about who owns a job, who can see an application, and whether a page is reading live data or demo-only data. By the final version, the system behaved more like one connected recruitment workflow rather than three disconnected role pages.

To here - no more than 10 pages including tables, charts, figures and diagrams.

Individual contribution and reflection

Each statement below is kept under 300 words.

Member 1 - Yuhang Fu - Authentication and account workflow

QM no: 231222431 Name: Yuhang Fu

Main contribution: I worked mainly on the login and account flow. This included role-based login, demo accounts, redirects after login, and the TA registration path. These parts were important because every later page depends on the user role being clear.

Reflective statement: This part taught me that authentication is also a user-experience problem. If the system sends a user to the wrong page, or lets the wrong role create an account, the rest of the workflow becomes confusing. I also learned to think more carefully about session state and role checks.

Member 2 - Jiajin He - TA application workflow

QM no: 231222682 Name: Jiajin He

Main contribution: I worked on the TA side, including the job list, job detail page, file upload and application submission. The final version uses live job data, real pagination, required CV/transcript upload, a confirmation step and duplicate submission protection.

Reflective statement: The main lesson for me was that a small button can create a real data problem. The repeated submit issue looked like a frontend detail at first, but it could create duplicate applications. Fixing it helped me connect UI state with backend validation.

Member 3 - Ruiqi Guo - MO job creation and screening

QM no: 231222257 Name: Ruiqi Guo

Main contribution: I worked on the MO workflow: creating jobs, viewing candidates for a job, filtering by status, and updating applications to interviewed, accepted or rejected. This connected MO-created vacancies with applications submitted by TA users.

Reflective statement: I learned that ownership rules need to be decided early. An MO should manage their own jobs, while a TA should see all open jobs. Mixing those two ideas caused some confusing behaviour, so this part helped me understand why permissions and data scope matter.

Member 4 - Shuai Na - HR review and workload support

QM no: 231220172 Name: Shuai Na

Main contribution: I worked on the HR/admin workspace, including candidate and job selection, AI decision support display, and the workload dashboard. I also helped make the workload table show applicant names instead of only internal IDs.

Reflective statement: This work showed me that dashboards need to be readable, not just technically correct. A table full of IDs may be useful for debugging, but it is not helpful for a reviewer. I became more aware of how backend data should be translated into user-facing information.

Member 5 - Leilei Tao - Testing, data, and documentation

QM no: 231222073 Name: Leilei Tao

Main contribution: I focused on seed data, endpoint notes, testing records and documentation support. I checked TA, MO and HR workflows through API calls, browser testing and build commands, especially after integration changes.

Reflective statement: I learned that testing a multi-role system means following the same data across pages. A feature can look correct on one page but still fail when another role needs to use its result. This made me value scenario-based testing more than checking pages separately.

Member 6 - Yi Zheng - UI consistency and MO review polish

QM no: 231222453 Name: Yi Zheng

Main contribution: I worked on UI consistency across TA, MO and HR pages. This included buttons, status labels, borders, backgrounds, upload feedback, disabled states, MO detail links and clearer attachment presentation.

Reflective statement: I learned that visual consistency is not only decoration. When button colours, disabled states or labels are inconsistent, users become unsure about the workflow. The difficult part was polishing the interface while keeping the underlying role logic unchanged.